k	value
1	"B"

الي بتقارن فيه.

2	"A"
5	"Z"

بضيفه بس لازم اقارنه اذا اكبر بيكون قبله واذا اصغر بعده. لان نمشي بالاولويه نقارن الارقام ماعلينا من الاسم

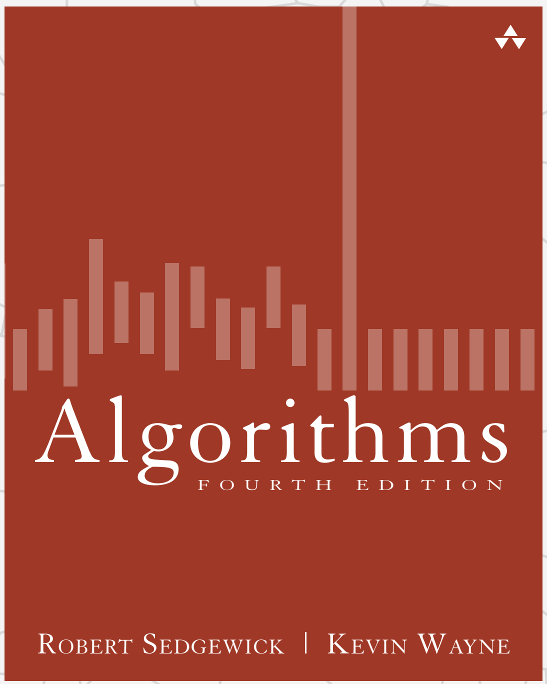
2	"A"	5	"Z"	1	"B"

FiFo → first in first out

بس ماراح نسويها

اولوية

2.4 PRIORITY QUEUES



http://algs4.cs.princeton.edu

- *API and elementary implementations*
- *binary heaps*
- *heapsort*
- *event-driven simulation*

1	"B"	2	"A"	5	"Z"

http://algs4.cs.princeton.edu

2.4 Priority Queues

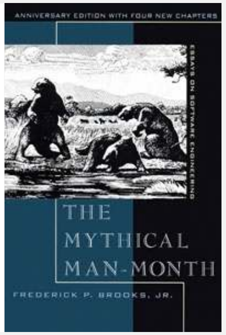
- *API and elementary implementations*
- *binary heaps*
- *heapsort*
- *event-driven simulation*

Collections

A collection is a data types that store groups of items.

data type	key operations	data structure
stack	PUSH, POP	*linked list, resizing array*
queue	ENQUEUE, DEQUEUE	*linked list, resizing array*
priority queue	INSERT, DELETE-MAX اكبر اولوية	*binary heap*
symbol table	PUT, GET, DELETE	*BST, hash table*
set	ADD, CONTAINS, DELETE	*BST, hash table*

" Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won't usually need your code; it'll be obvious." — *Fred Brooks*



Priority queue

Collections. Insert and delete items. Which item to delete?

Stack. Remove the item most recently added.

Queue. Remove the item least recently added.

Randomized queue. Remove a random item.

Priority queue. Remove the largest (or smallest) item.

operation	*argument*	*return value*	
insert	P		
insert	Q		
insert	E		
remove max		Q	
insert	X		لانه اكبر واحد
insert	A		
insert	M		
remove max		X	
insert	P		
insert	L		
insert	E		
remove max		P	

Priority queue API

Requirement. Generic items are `Comparable`.

Key must be Comparable
(bounded type parameter)

Min

```
public class MaxPQ<Key extends Comparable<Key>>
```

	`MaxPQ()`	*create an empty priority queue*
	`MaxPQ(Key[] a)`	*create a priority queue with given keys*
`void`	`insert(Key v)`	*insert a key into the priority queue*
`Key`	`delMax()` → deQueue	*return and remove the largest key*
`boolean`	`isEmpty()`	*is the priority queue empty?*
`Key`	`max()`	*return the largest key*
`int`	`size()`	*number of entries in the priority queue*

Priority queue applications

- Event-driven simulation. [customers in a line, colliding particles]
- Numerical computation. [reducing roundoff error]
- Data compression. [Huffman codes]
- Graph searching. [Dijkstra's algorithm, Prim's algorithm]
- Number theory. [sum of powers]
- Artificial intelligence. [A* search]
- Statistics. [online median in data stream]
- Operating systems. [load balancing, interrupt handling]
- Computer networks. [web cache]
- Discrete optimization. [bin packing, scheduling]
- Spam filtering. [Bayesian spam filter]

Generalizes: stack, queue, randomized queue.

Priority queue elementary implementations

Challenge. Implement all operations efficiently.

time complex 1 2 3 do not serach

implementation	insert	del max	max
unordered array	1	N	N
ordered array	N	1	1
goal	$\log N$	$\log N$	$\log N$

لان بمر على كل ال node.

order of growth of running time for priority queue with N items

o(n)
insert
constant
orderd data

http://algs4.cs.princeton.edu

2.4 Priority Queues

- API and elementary implementations
- binary heaps
  complete binary tree
- heapsort
- event-driven simulation

Complete binary tree

Binary tree. Empty or node with links to left and right binary trees. 2 child

Complete tree. Perfectly balanced, except for bottom level.

heaps: Max heap : Node must be grater of the Childeren.

All node shift to right

ماتسوي مقارنه بين اليمين واليسار

تسوي مقارنه ال node مع ال Parent

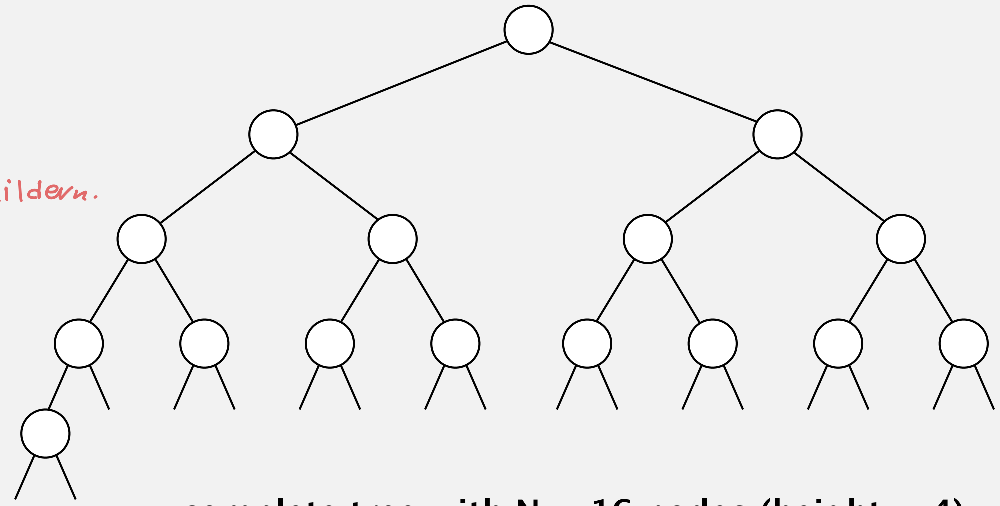
Min : heap : node must be less of the Childern.

alwas remove root.

$(\log_2 n)$

Input : max, delet, insert, min

space : $O(n)$



complete tree with N = 16 nodes (height = 4)

Property. Height of complete tree with N nodes is $\lfloor \lg N \rfloor$.

Pf. Height increases only when N is a power of 2.

سويناها Queue عادية

Q5. What does each removeMin call return within the following sequence of **priority queue** (min heap) ADT operations. Each node in the binary tree stores the priority key (integer) and the value (character).

insert(5, A),
insert(4, B),
insert(7, F),
insert(1, D),
removeMin( ),
insert(3, J),
insert(6, L),
يعتمد على ال Key ← removeMin( ),
removeMin( ),
insert(8, G),
removeMin( ),
insert(2, H),
removeMin( ),
removeMin( )

~~(1, D)~~	(4, B)	(5, A)	(7, F)

ليش لان لها اولوية

return: (1, D), (3, j), (4, B), (5, A), (2, H), (6, L)

سوينا كذا علشان ماتلخبط

~~(3, j)~~	~~(4, B)~~	~~(5, A)~~	(6, L)	(7, F)	(8, G)	(2, H)

المفروض كذاهم

return: (3, j), (4, B), (5, A)

~~(2, H)~~	~~(6, L)~~	(7, F)	(8, G)

return: (2, H), (6, L)

A complete binary tree in nature

Binary heap representations

Binary heap. Array representation of a heap-ordered complete binary tree.

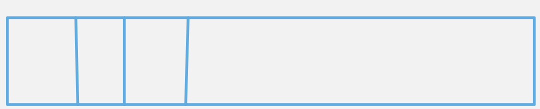
Heap-ordered binary tree.

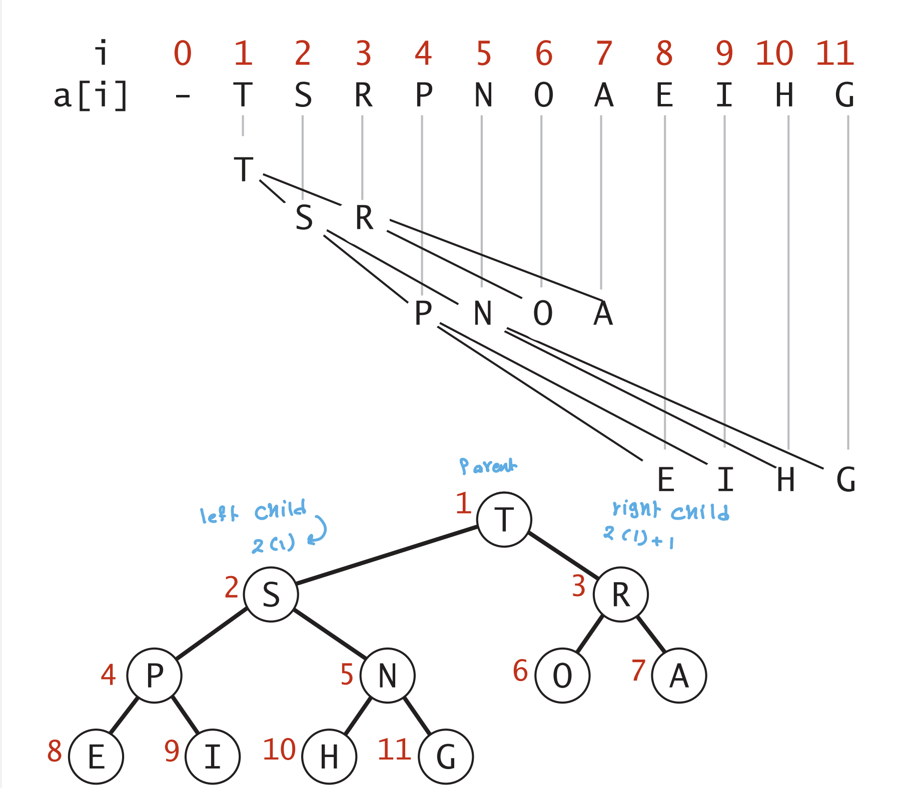
- Keys in nodes.
- Parent's key no smaller than children's keys.

use

Array representation. b/c it is complete

- Indices start at 1.
- Take nodes in level order.
- No explicit links needed!





Heap representations

Binary heap properties

الاب اكبر من اولادة

Proposition. Largest key is a[1], which is root of binary tree.

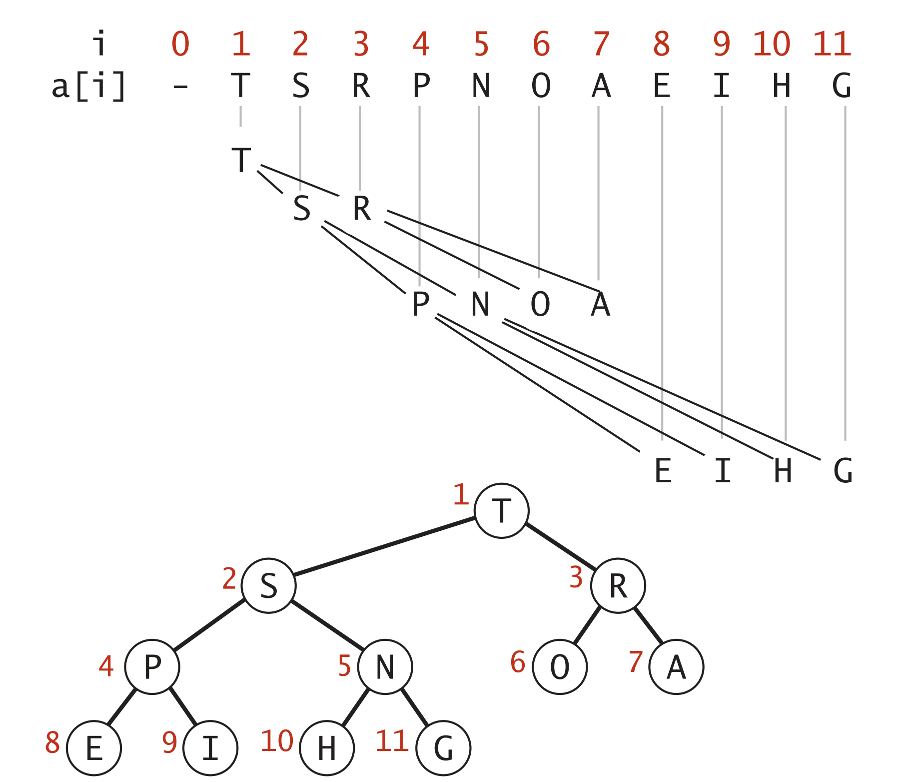
Proposition. Can use array indices to move through tree.

- Parent of node at k is at k/2. $11/2 = 5 \rightarrow$ الاب
- Children of node at k are at 2k and 2k+1.

heap $\xrightarrow{\text{by deflt}}$ Max heap ($N \geq L, R$)

if Minssion Min heap ($N \leq L, R$)

$R \not\leftrightarrow L$ No realtion don't care



Heap representations

Binary heap demo

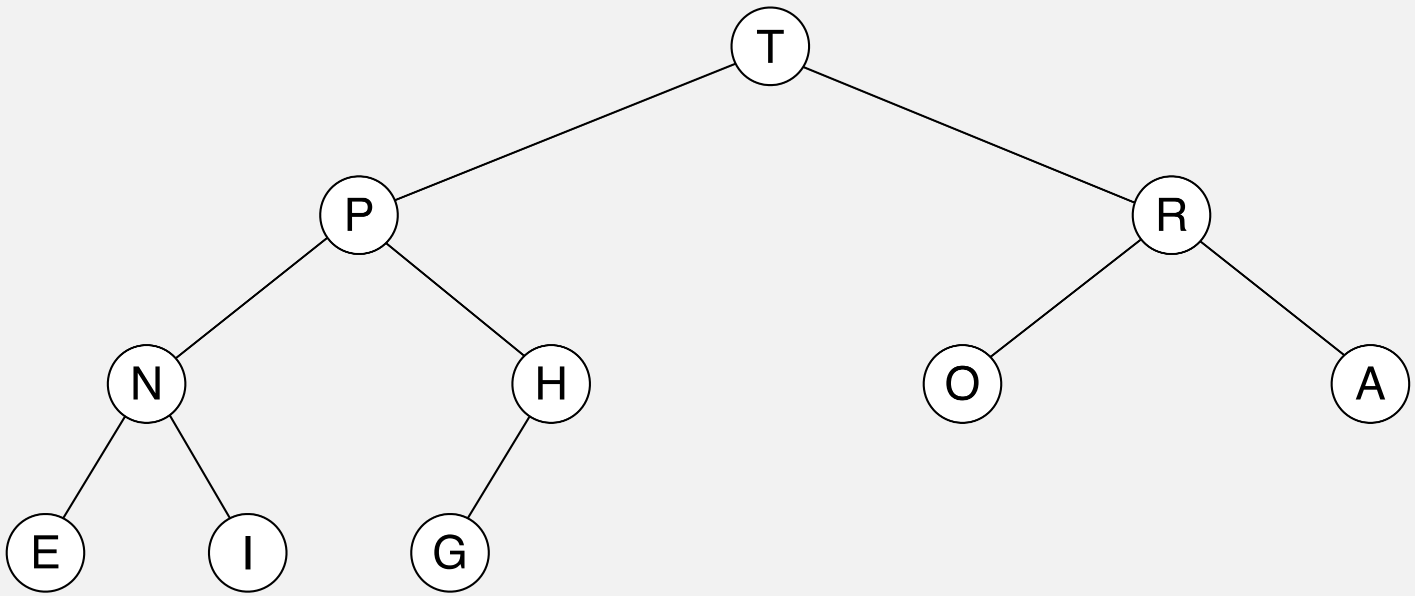
Insert. Add node at end, then swim it up.

Remove the maximum. Exchange root with node at end, then sink it down.

يحذف

heap ordered

لازم مرتبه



T	P	R	N	H	O	A	E	I	G	

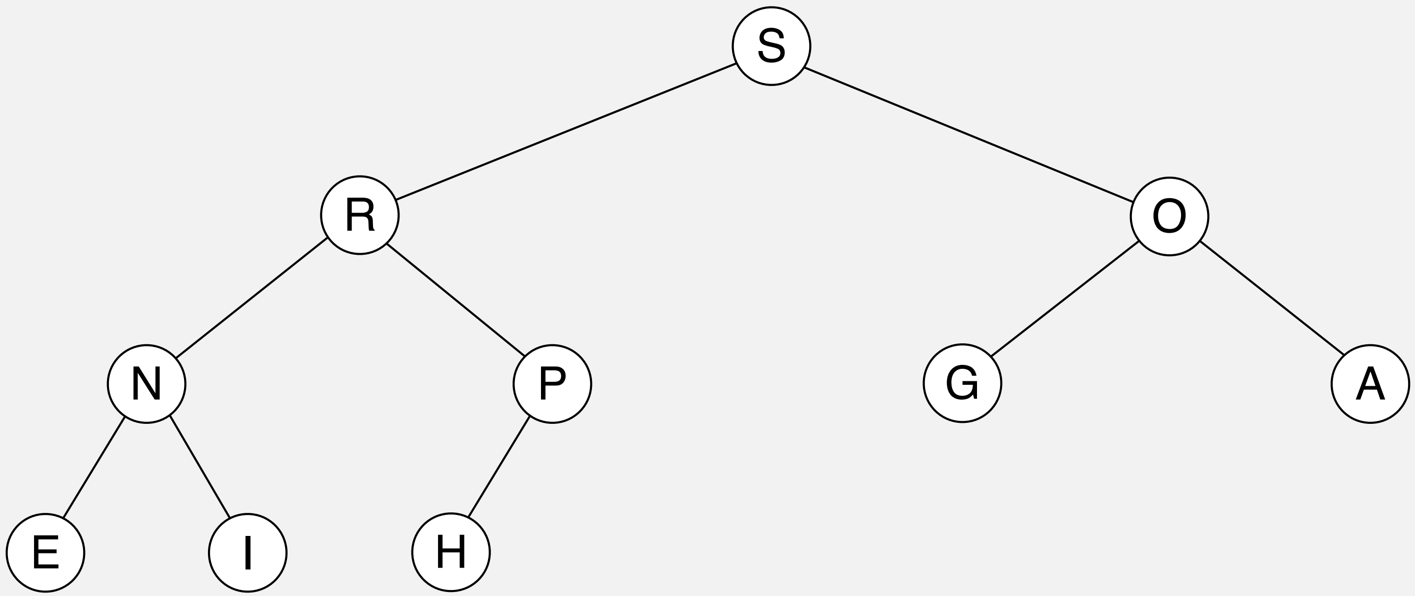
Binary heap demo

Insert. Add node at end, then swim it up.

Remove the maximum. Exchange root with node at end, then sink it down.

heap ordered

Maximum heap



S	R	O	N	P	G	A	E	I	H	

Promotion in a heap

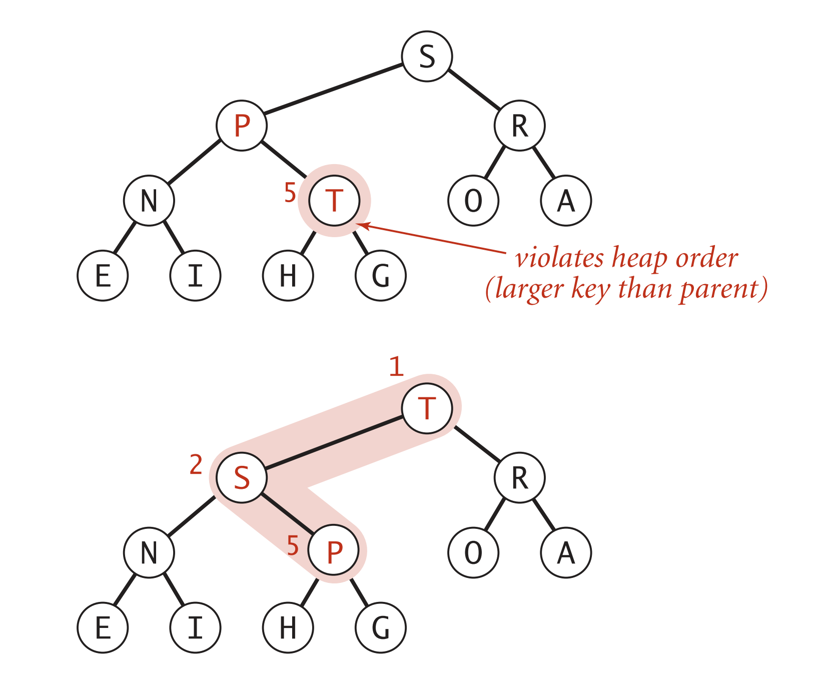
Scenario. Child's key becomes larger key than its parent's key.

To eliminate the violation:

- Exchange key in child with key in parent.
- Repeat until heap order restored.

```
private void swim(int k)
{
   while (k > 1 && less(k/2, k))
   {
      exch(k, k/2);
      k = k/2;
   }
}
```

parent of node at k is at k/2



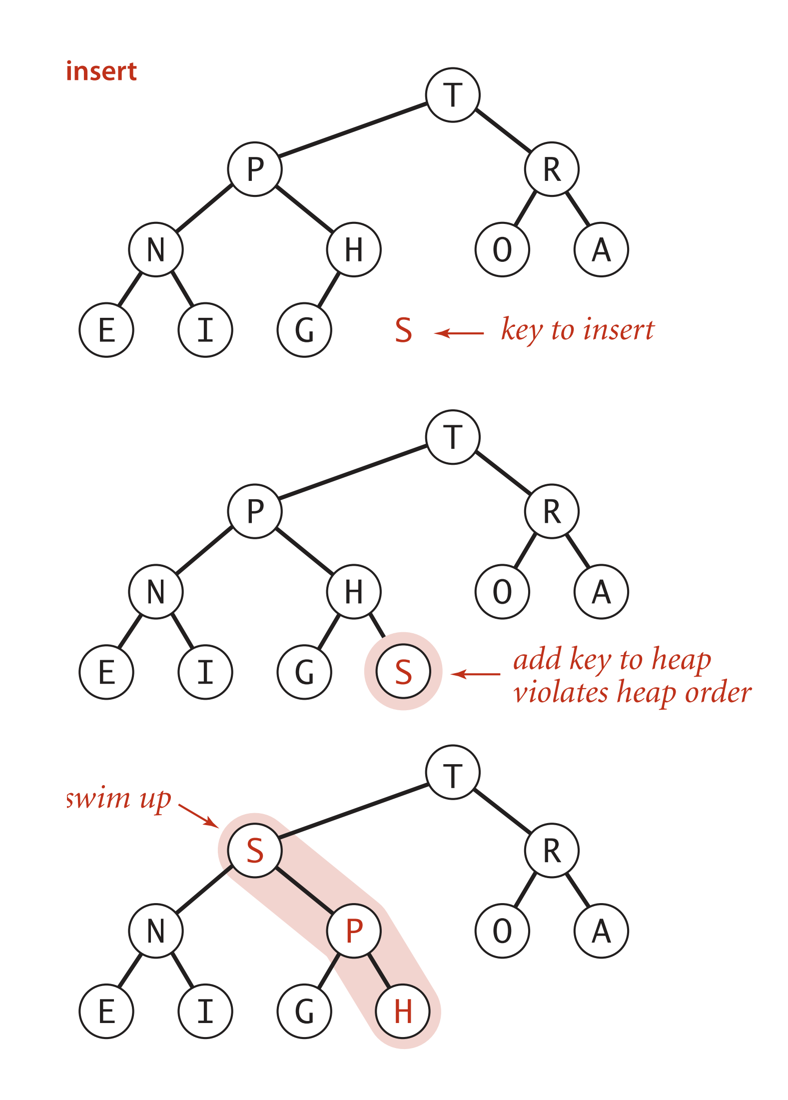
Peter principle. Node promoted to level of incompetence.

Insertion in a heap

Insert. Add node at end, then swim it up.

Cost. At most $1 + \lg N$ compares.

```
public void insert(Key x)
{
   pq[++N] = x;
   swim(N);
}
```



Demotion in a heap

Scenario. Parent's key becomes smaller than one (or both) of its children's.

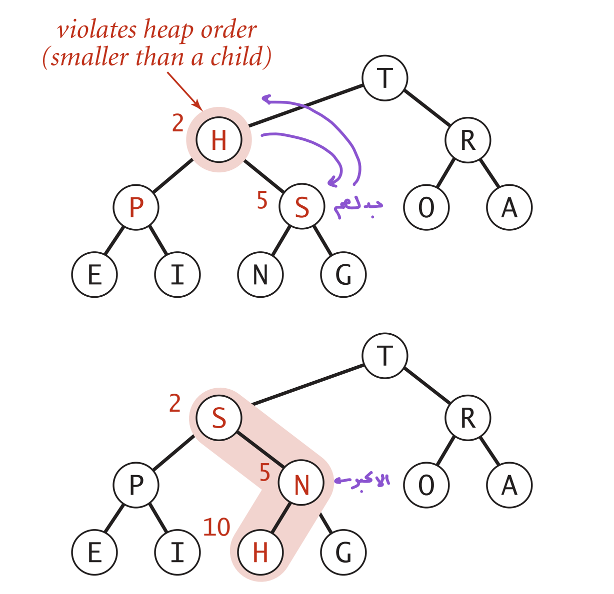
To eliminate the violation:

why not smaller child?

- Exchange key in parent with key in larger child.
- Repeat until heap order restored.

```
private void sink(int k)
{
   while (2*k <= N)
   {
      int j = 2*k;
      if (j < N && less(j, j+1)) j++;
      if (!less(k, j)) break;
      exch(k, j);
      k = j;
   }
}
```

children of node at k are 2k and 2k+1



Top-down reheapify (sink)

بفيف منا دتحت واسوي Swime UP اطلعه فوق
ولمن احذف ، احذف الـ node واضيفها مكان root وبرتبها.

Power struggle. Better subordinate promoted.

Delete the maximum in a heap

Delete max. Exchange root with node at end, then sink it down.

Cost. At most $2 \lg N$ compares.

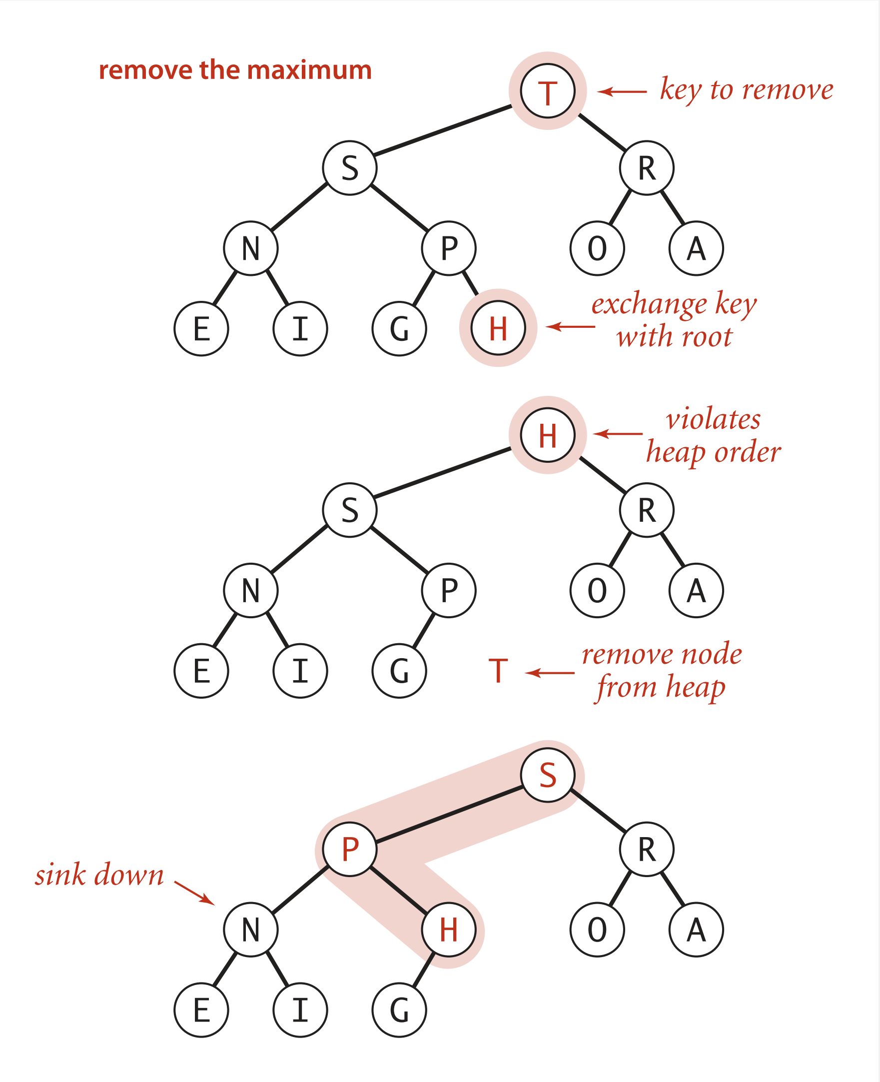
```
public Key delMax()
{
   Key max = pq[1];
   exch(1, N--);
   sink(1);
   pq[N+1] = null;
   return max;
}
```

Delet root

Index root ←

↳ of last node

prevent loitering



Binary heap: Java implementation

```
public class MaxPQ<Key extends Comparable<Key>>
{
   private Key[] pq;
   private int N;

   public MaxPQ(int capacity)
   {  pq = (Key[]) new Comparable[capacity+1];  }

   public boolean isEmpty()
   {   return N == 0;   }
   public void insert(Key key)
   public Key delMax()
   {   /* see previous code */  }

   private void swim(int k)
   private void sink(int k)
   {   /* see previous code */  }

   private boolean less(int i, int j)
   {   return pq[i].compareTo(pq[j]) < 0;  }
   private void exch(int i, int j)
   {   Key t = pq[i]; pq[i] = pq[j]; pq[j] = t;  }
}
```

how match element we have.

insert

Delet

fixed capacity (for simplicity)

PQ ops

heap helper functions

array helper functions

Priority queues implementation cost summary

one element

implementation	insert	del max	max
unordered array	1	N	N
ordered array	N	1	1
binary heap	$\log N$	$\log N$	1

order-of-growth of running time for priority queue with N items